the right notion is a choice of tied parameters such that the model is, aggregated over positions, optimal for the present task when conditioned on a fixed past. That is, forward passes do not compute features for the future if they can compute other features more beneficial to the present.

Definition 3. *The parameters $\tilde{\boldsymbol{\theta}} \in \Theta$ are **(tied-)myopic** if they satisfy*

$$\tilde{\boldsymbol{\theta}} \in \arg\min_{\boldsymbol{\theta}\in\Theta} \sum_{i=1}^{n} \ell_i(\tilde{\boldsymbol{\theta}}, \ldots, \tilde{\boldsymbol{\theta}}, \boldsymbol{\theta}). \tag{3}$$

*The **(tied) myopia gap** is then defined analogously to Definition 2.*

The *breadcrumbs hypothesis* states that the myopia gap is small—near-optimal performance can be attained even when each forward pass is computing features relevant to only its own immediate inference task, with no regard to pre-caching for the future.

If the breadcrumbs hypothesis does not hold, we say that the model is *pre-caching*. It is important to remember that the ℓ_i depend on a choice of transformer model G and dataset $\mathcal{D}$. That is, breadcrumbs and pre-caching are properties of the model architecture and the data considered as a whole.

Although a small myopia gap reveals that one can do just as well without pre-caching, it does not say much about any specific model. To measure pre-caching within a given model, we examine the extent to which its parameters violate the myopia constraints.

Definition 4. *The **(tied) local myopia bonus** at $\boldsymbol{\theta}^* \in \Theta$ is*

$$\hat{c}(\boldsymbol{\theta}^*) := \max_{\boldsymbol{\theta}\in\Theta} \sum_{i=1}^{n} \left(\ell_i(\boldsymbol{\theta}^*, \ldots, \boldsymbol{\theta}^*) - \ell_i(\boldsymbol{\theta}^*, \ldots, \boldsymbol{\theta}^*, \boldsymbol{\theta})\right) \geq 0.$$

For further interpretation of the myopia gap and myopia bonus, see Appendix A.

3.4 Myopic gradient descent

Our heuristic remark in Section 3.2, that the off-diagonal gradient terms are responsible for pre-caching, is justified by Theorem 13 below. It states that, given certain regularity conditions on the loss terms ℓ_i, performing gradient descent with the off-diagonal terms removed results in a myopic model in the sense of Definition 3. We call this *myopic descent*.

For myopic descent to be stable in the tied-weights case, we need, roughly speaking, for the model to depend more on the parameters associated with the present forward pass than those from the past. This is a plausible condition—dependence on the past is mediated purely by the attention mechanism, while the present forward pass depends on both attention and feedforward parameters. The precise condition we use is **forward bias** (Definition 11); see Appendix C for details and the proof of the theorem.

Theorem 13. *Let $f(\tilde{\boldsymbol{\theta}}, \boldsymbol{\theta}) := \sum_{i=1}^{n} \ell_i(\tilde{\boldsymbol{\theta}}, \ldots, \tilde{\boldsymbol{\theta}}, \boldsymbol{\theta})$. If f is forward-biased, σ-strongly convex, and L-smooth, then, for some step size $\eta > 0$, the iterates of myopic descent with tied weights*

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla_{\boldsymbol{\theta}} f(\tilde{\boldsymbol{\theta}}, \boldsymbol{\theta})\big|_{\tilde{\boldsymbol{\theta}}=\boldsymbol{\theta}=\boldsymbol{\theta}^{(t)}}$$

converge to $\tilde{\boldsymbol{\theta}} \in \Theta$ satisfying the myopia constraints of Equation 3.

4 Synthetic data experiments

4.1 The $\mathcal{D}_p$ task

To demonstrate a simple example where the transformer model learns to pre-cache (and thus the myopia gap is large), we construct the following synthetic dataset.

Definition 5. *The data distribution $\mathcal{D}^N_{p,a,b}$ is defined as the joint distribution of real-valued random variables $(\mathrm{x}_n)_{n=1}^N, (\mathrm{y}_n)_{n=1}^N, (\mathrm{z}_n)_{n=1}^N$ where, for each n,*

- $x_n \sim \mathcal{N}(0,1)$ *(standard Gaussian)*
- $z_n \sim \text{Ber}(p)$ *(Bernoulli with probability p)*
- $y_n = z_n \sum_{i=1}^{a} \sin(b x_{n-i}) + (1 - z_n) x_n$

and $\{x_n\}_{n\in\mathbb{N}} \cup \{z_n\}_{n\in\mathbb{N}}$ are mutually independent. In our experiments, we always set the parameters $a = b = 10$ and $N = 64$, so for convenience notate $\mathcal{D}_p := \mathcal{D}^{64}_{p,10,10}$.

The intuition is that a transformer regression model G trained on $\mathcal{D}_p$ would benefit from pre-caching $\sin(b x_n)$ during its forward pass at position n, even though this computation is irrelevant to its task of predicting y_n. One simple strategy that makes use of this pre-caching is Algorithm 1. [4]

The motivation for the Bernoulli variables z_i is that, as p decreases, the expected first time when $\sin(b\mathbf{x}_n)$ becomes useful advances further into the future. In addition, when p is sufficiently small, the probability $(1-p)^a$ that the value $\sin(b x_n)$ is never useful at all becomes non-negligible. We will show that, even in this case, the transformer model learns to pre-cache.

Investigating myopia. Suppose that we train a myopic model (Section 3.4) on the same task. Since this model lacks off-diagonal gradient terms, we do not expect it to learn to pre-cache $\sin(b x_n)$ at position n. One possible strategy that does not use pre-caching is Algorithm 2. We expect this brute force algorithm to perform significantly worse given the same parameter count—it computes an a-dimensional nonlinear function within a single layer, while each layer of Algorithm 1 computes only scalar nonlinear functions.[5]

Algorithm 1 Pre-caching algorithm

At position n,
input x_n, z_n
layer 1 compute $F_n := \sin(b x_n)$
layer 2 read F_{n-i} for $i = 1, \ldots, a$.
layer 2 compute
$\hat{y}_n := z_n \sum_{i=1}^{a} F_{n-i} + (1 - z_n) x_n$.
return $\hat{y}_n$.

Algorithm 2 Brute force algorithm

At position n,
input x_n, z_n
layer 1 compute $\emptyset$
layer 2 read x_{n-i} for $i = 1, \ldots, a$.
layer 2 compute
$\hat{y}_n := z_n \sum_{i=1}^{a} \sin(b x_{n-i}) + (1 - z_n) x_n$.
return $\hat{y}_n$.

4.1.1 Evaluation: linear probing

To determine if the transformer model is computing $\sin(b x_n)$ at position n, we fit linear probes on the hidden states. We additionally compute the correlations between $\sin(b x_n)$ and each individual dimension (i.e., each neuron) of each hidden state.[6] See Section D.1.1 for details.

[4] We think of each transformer layer as a **read** operation, performed by the attention mechanism, followed by a **compute** operation, performed by the feedforward block.

[5] For example, Shen et al. (2022) provide upper bounds on MLP error that degrade exponentially in dimensionality given a fixed parameter and layer count.

[6] Note that, in order for linear probing to be meaningful, we must first ensure that there is no pre-existing linear relationship between the inputs and the quantities we are probing for. Since the $(x_n, z_n)_n$ are mutually independent, this follows from Lemma 15, stating that x_n and $\sin(b x_n)$ have near-zero correlation for large enough b. In our experiments, we set $b = 10$, in which case $\rho(x_n, \sin(b x_n)) < 10^{-20}$. In other words, there is low predictive $\mathcal{V}$-information from the inputs to the target $\sin(b x_n)$, where $\mathcal{V}$ is the class of linear models (Xu et al., 2020).

4.1.2 Results for $\mathcal{D}_p$

For varying p, we train two-layer transformer models with embedding dimensions of 128 on $\mathcal{D}_p$ using using both ordinary and myopic gradient descent. Full architecture and training details are provided in Section D.1.

Examining the performance of each linear probe against $\sin(b\mathrm{x}_{n-i})$ for varying i, we find strong evidence that the transformer model with vanilla training is indeed pre-caching $\sin(b\mathrm{x}_n)$, possibly in order to implement Algorithm 1. Indeed, in Figure 2,

- The zeroth hidden state (i.e., the sum of the input and position embeddings) at position n is correlated with only x_n.
- The first hidden state is correlated with $\sin(b\mathrm{x}_n)$ but not correlated with any $\sin(b\mathrm{x}_{n-i})$ for $i > 0$.
- The second hidden state (immediately before the output unembedding) is correlated with $\sin(b\mathrm{x}_{n-i})$ for each $0 \leq i \leq a$.

Further, looking at the per-neuron correlations in Figure 3, we see that $\sin(b\mathrm{x}_{n-i})$ for $1 \leq i \leq a$ are all correlated with a single 1-d subspace of the second hidden state (they share the same striping pattern); this is the subspace storing $\sum_{i=1}^{a} \sin(b\mathrm{x}_{n-i})$. Meanwhile, $\sin(b\mathrm{x}_n)$, as well as many of the x_{n-i}, are located in various other 1-d subspace of the second hidden state; these terms are all left over in the residual stream from previous layers, and are cleaned up only by the output unembedding.

p	**Vanilla**	**Myopic**
0.01	0.096	1.10
0.1	0.016	0.97
0.3	0.0030	1.03
1.0	0.0074	1.26

Table 1: Normalized Huber loss $\mathcal{L}/p$ for vanilla and myopic models trained and evaluated on $\mathcal{D}_p$ for each p in our synthetic setting. For reference, the trivial model that always outputs zero attains a Huber loss of **1.26**.

On the other hand, in Table 1, the myopic models perform significantly worse. The per-neuron correlations in Figure 3 suggest that the myopic model may be implementing a crude approximation of Algorithm 2. This suggests that the synthetic setting has an inherently high *myopia gap*—it is impossible for the transformer model to do well without pre-caching.

4.2 Multiplication

In addition to the above $\mathcal{D}_p$ synthetic data setting, we also measure the myopia gap on the task of natural number multiplication. In particular, we find evidence suggesting that pre-caching is responsible for model computation on filler tokens, in the sense of Pfau et al. (2024). See Appendix D.3 for details.

5 Natural language experiments

5.1 GPT-2's myopia gap

In order to measure the extent to which transformer models learn to pre-cache on natural language data, we estimate both the myopia gap (Definition 3) in this setting as well as the local myopia bonus (Definition 4) of a transformer model with vanilla pre-training. Experiments in this subsection use the 124M-parameter GPT-2 architecture; see Table 4 in Appendix D for configuration details.

We train all models (vanilla and myopic) from random initialization for one epoch on 4.6M sequences from the MS MARCO dataset (Nguyen et al., 2016), truncated to length 64. To estimate the local myopia bonus of the vanilla model, we train another model from random initialization with the same architecture, but with past hidden states sourced from the frozen

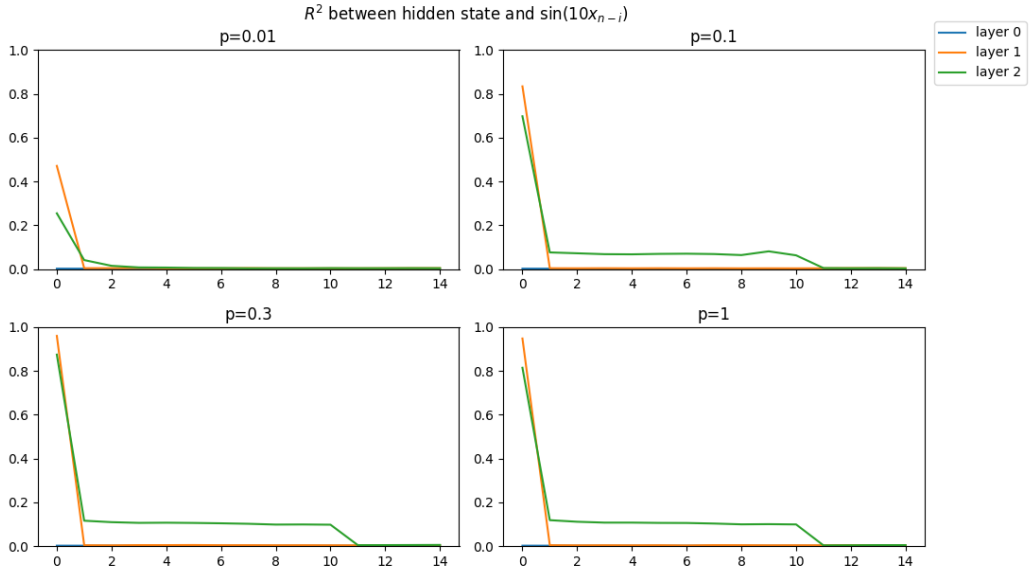


Figure 2: Empirical R^2 between linear probes fit on each layer of vanilla transformer models trained on $\mathcal{D}_p$ for $p \in \{0.01, 0.1, 0.3, 1\}$ to targets $\sin(bx_{n-i})$. Computed over 50 000 samples from $\mathcal{D}_1$.

vanilla model during both training and evaluation.[7] See Appendix B for implementation details.

As baseline, we also train a "transformer bigram" model, a model with an identical architecture but all off-diagonal key/value states zeroed out.

5.1.1 GPT-2 results

From Table 2, the estimated myopia gap in this setting is $3.40 - 3.28 = \mathbf{0.12}$ cross entropy, while the local myopia bonus of the vanilla model is $3.28 - 3.26 = \mathbf{0.02}$.

The nonzero myopia gap suggests that pre-caching may provide a small positive benefit. Indeed, in Figure 4, we see that the myopic model outperforms the vanilla model at the beginning of the sequence, since it can allocate all compute to next-token prediction, but quickly falls behind as the length of the past increases, since it suffers from a lack of pre-cached information from earlier forward passes.[8]

However, note that this gap is much smaller than that between the vanilla model and the transformer bigram model (Table 2). That is, the myopic model is still able to leverage past information (breadcrumbs) to a significant extent, even if they optimized only for the present inference task. That the local myopia gap is near zero further supports this direction—the model learned through vanilla training does not trade off significantly between features useful for the present and pre-caching for the future.

Model	Cross-entropy
Vanilla	3.28
Myopic	3.40
Local myopic	3.26
Transformer bigram	5.33

Table 2: Validation cross-entropy loss obtained by GPT-2 with vanilla and myopic training

[7] Note that this "local myopic" model attains slightly *better* performance than the vanilla model; each forward pass can focus purely on next-token prediction, since past hidden states are supplied by a separate model.

[8] We use a sliding window over PG-19 (Rae et al., 2019) samples, which comprise longer sequences, in order to reduce noise in our per-position loss estimates. The distribution shift between MS MARCO and PG-19 does not affect the relative comparison of myopic and vanilla GPT-2.